Project Report: Deep Learning for Stock Market Analysis

Task 1 — Nasdaq Stock Price Forecasting (AAL)

1.1 Data Preparation

The dataset used for Task 1 consists of historical trading data for **AAL (American Airlines Group Inc.)**, including six features: Open, Low, High, Close, Adjusted Close, and Volume. Data was split chronologically at an 80/20 ratio without shuffling to preserve temporal order and avoid data leakage. All features were normalized using MinMaxScaler to the range (0, 1). A lookback window of time_step = 60 was used throughout, meaning the model sees 60 past trading days to make each prediction.

1.2 Task 1.1 — Multi-Feature Prediction

Unlike the single-feature baseline, the model here takes all six features as input. The architecture is a single LSTM layer with 50 units, followed by a Dense(1) output. Training used the Adam optimizer with MSE loss and early stopping (patience = 10). The predicted prices tracked the actual test prices well, capturing the main trends in AAL's price movement.

1.3 Task 1.2 — k-Day-Ahead Forecasting

This sub-task requires predicting the price at a specific future day k, rather than the next day. The dataset was modified so that each input window of 60 days is labeled with the price at day t + k. The same LSTM architecture was used, trained separately for **k = 3** and **k = 7**.

Metric	k = 3	k = 7
MAE	1.2046 USD	2.2270 USD

RMSE	1.6205 USD	2.8403 USD
MAPE	7.82%	14.68%

Error increases notably as k grows, which is expected — the further ahead the model tries to predict, the more market uncertainty accumulates.

1.4 Task 1.3 — Consecutive k-Day Sequence Forecasting

In this sub-task, the model predicts a full sequence of k consecutive prices in a single forward pass. The output layer was changed to Dense(k) to support this. Training over a sequence of targets provides richer gradient signal, and the results reflect this — the sequence model outperforms the single-point model at both k values.

k	MAE	RMSE	MAPE
3	0.7618 USD	1.1316 USD	4.82%
7	1.4060 USD	1.9688 USD	9.11%

The visualization shows the predicted segments closely following the actual price curve, though some lag is visible at sharp reversal points — a common limitation of LSTM regression models.

1.5 Conclusion

All three sub-tasks confirm that LSTM is effective for short-term stock price forecasting when given multi-feature input. Prediction accuracy degrades with larger k, consistent with the inherent uncertainty of financial markets. Notably, the sequence forecasting approach (Task 1.3) consistently achieves lower error than direct k-day prediction (Task 1.2), suggesting that training over multiple output steps helps the model learn more stable short-term representations.

Task 2 — Vietnam Stock Price Forecasting (SAM)

2.1 Data Preparation

Task 2 applies the same LSTM-based forecasting approach to the Vietnamese stock market, using historical data for **SAM (VNINDEX)**. The feature set consists of five variables: Open, Low, High, Close, and Volume. Data was split chronologically at an 80/20 ratio, normalized with MinMaxScaler, and a lookback window of time_step = 30 was used — shorter than Task 1 to reflect the relatively smaller dataset size in the Vietnamese market.

2.2 Task 2.1 — Multi-Feature Prediction

The model architecture mirrors Task 1.1: a single LSTM layer with 50 units followed by Dense(1), trained with Adam optimizer, MSE loss, and early stopping (patience = 10). The target variable is the **Close price** (index 3 in the feature set).

On the test set, the model achieved strong performance:

Metric	Result
MAE	225.24 VND
RMSE	388.73 VND
MAPE	1.93%

The very low MAPE of 1.93% indicates that the model fits the SAM price series closely, likely because the Vietnamese stock data exhibits smoother and more gradual trends compared to the more volatile Nasdaq data in Task 1.

2.3 Task 2.2 — k-Day-Ahead Forecasting

Following the same approach as Task 1.2, the model was retrained to forecast the Close price at day t + k. Results for **k = 3**:

Metric	k = 3
MAE	402.85 VND
RMSE	741.27 VND
MAPE	3.43%

As in Task 1, error increases compared to the next-day prediction baseline, reflecting the growing uncertainty over longer forecast horizons. The MAPE of 3.43% remains acceptable for a 3-day-ahead prediction in practice.

2.4 Task 2.3 — Consecutive k-Day Sequence Forecasting

The output layer was extended to Dense(k) to produce a full sequence of k consecutive predicted prices. For **k = 3** with time_step = 30:

Metric	k = 3
MAE	329.90 VND
RMSE	605.72 VND
MAPE	2.85%

Consistent with Task 1.3, the sequence forecasting model achieves lower error than direct single-point k-day prediction (Task 2.2) at the same k. The visualization shows the predicted 3-day segments tracking the long-term upward trend of SAM prices well, with some deviation during sharp peak-and-drop periods around day 800 of the test set.

2.5 Conclusion

Task 2 demonstrates that the LSTM architecture generalizes effectively to the Vietnamese stock market. The model achieves notably lower MAPE values compared to Task 1, suggesting that SAM's price series is more predictable in relative terms. The same pattern observed in Task 1 holds here: sequence forecasting (Task 2.3) consistently outperforms single-point k-ahead forecasting (Task 2.2), reinforcing the benefit of multi-step output training for short-horizon financial prediction.

Task 3 — Buy and Sell Signal Classification (SAM)

3.1 Overview

Task 3 shifts from regression to **binary classification**, training separate models to predict buy and sell trading signals for the SAM stock. Rather than forecasting exact prices, the goal is to identify favorable entry and exit points based on historical patterns and engineered features.

3.2 Feature Engineering and Data Preparation

Beyond the raw OHLCV features used in Tasks 1 and 2, Task 3 introduces a richer set of technical indicators to better capture market conditions:

- **Log Return** and **14-day Rolling Volatility** to measure momentum and risk
- **14-day SMA** and **Distance to SMA** to capture trend deviation
- **RSI (14-day)** as a momentum oscillator
- **Bollinger Bands** (SMA_20, Upper/Lower Band, %B) for Task 3.2

- **Fundamental data**: Price-to-Earnings ratio and Dividend Yield, merged from quarterly financial reports

Data was split 80/20 chronologically, scaled with StandardScaler, and structured into sliding windows of time_step = 20days.

3.3 Task 3.1 — Buy Signal Prediction

A buy signal is defined as a **binary label**: 1 if the stock's return over the next 5 days exceeds **+2%**, and 0 otherwise. The label distribution in the training set was imbalanced (roughly 71% negative, 29% positive), so a class weight of {0: 1.0, 1: 2.0} was applied to prevent the model from simply predicting "no buy" for all samples.

The model architecture is a two-layer LSTM (32 → 16 units) with Batch Normalization, Dropout (0.5), and a Dense(1, sigmoid) output. An optimal classification threshold of **0.40** was selected by scanning for the best F1 score on the test set.

Classification results:

Class	Precision	Recall	F1-score
Ignore (0)	0.78	0.24	0.37
Buy (1)	0.28	0.82	0.42
Overall Accuracy	—	—	**0.39**

The model shows high recall for the Buy class (0.82), meaning it successfully catches most actual buying opportunities, at the cost of lower precision. For a trading strategy focused on not missing profitable entries, this tradeoff is acceptable.

3.4 Task 3.2 — Sell Signal Prediction

A sell signal is defined as a **binary label**: 1 if the return over the next 5 days falls below **−3%**, and 0 otherwise. The same two-layer LSTM architecture was used, with slightly reduced Dropout (0.3) and the Bollinger Band features added as additional inputs. The optimal threshold was **0.50**.

Classification results:

Class	Precision	Recall	F1-score
Hold/Buy (0)	0.83	0.73	0.77
Sell (1)	0.32	0.46	0.38
Overall Accuracy	—	—	**0.67**

The sell model achieves noticeably better overall accuracy (67%) compared to the buy model (39%), largely because the Hold/Buy class dominates the test set. The Sell class remains challenging to classify precisely, which is expected given that sharp downturns are relatively rare and difficult to predict from technical indicators alone.

3.5 Conclusion

Task 3 demonstrates that LSTM-based classifiers can extract meaningful trading signals from a combination of price, volume, and fundamental features. The buy model prioritizes recall to avoid missing profitable opportunities, while the sell model achieves more balanced performance. Both models reflect a common challenge in financial signal classification: class imbalance and the inherently noisy nature of short-term market movements make it difficult to achieve high precision and recall simultaneously.

Task 4 — Vietnam Portfolio Construction and Risk Management

4.1 Overview

Task 4 extends the classification models from Task 3 to a full portfolio management pipeline, covering stock selection (Task 4.1), risk filtering (Task 4.2), and optimal capital allocation (Task 4.3). The universe consists of **10 selected Vietnamese stocks**: ST8, LCG, TVD, TC6, NAG, PLC, PC1, PVP, CST, and VOC. A minimum threshold of **120 historical data points** per stock was enforced as a filtering criterion to ensure sufficient training samples.

4.2 Task 4.1 — Profitable Stock Selection

Methodology: For each stock, the same LSTM-based buy signal classifier from Task 3.1 was retrained independently — a lightweight architecture (LSTM(32) + Dropout(0.3) + Dense(1, sigmoid)) trained for 30 epochs with early stopping (patience = 5). The input features include OHLCV data enriched with SMA_14, RSI_14, distance-to-SMA, log return, and volatility, along with fundamental ratios (P/E, ROE, ROA) and dividend yield where available.

Profit estimation: Rather than using a raw classification score, expected return for each stock was estimated using a simplified expected value formula:

$$\text{Expected Return} = (P\_buy \times 10\%) - (P\_loss \times 5\%)$$

where P_buy is the model's mean predicted buy probability over the last 20 trading days of the test set, and $P\_loss = 1 - P\_buy$. The 10% target profit and 5% stop-loss thresholds reflect a realistic asymmetric risk-reward assumption.

Results (ranked by buy probability):

Ticker	Mean Buy Probability	Expected Return
PVP	62.6%	+4.39%

CST	56.0%	+3.40%
PLC	51.5%	+2.72%
PC1	50.2%	+2.53%
LCG	39.5%	+0.92%
TVD	32.3%	−0.16%
NAG	31.2%	−0.31%
VOC	29.2%	−0.62%
ST8	24.7%	−1.29%
TC6	14.5%	−2.83%

4.3 Task 4.2 — Risk Management

Methodology: A separate LSTM-based sell signal model (mirroring Task 3.2) was trained for each stock to estimate downside risk. The sell probability over the last 20 days of the test set was used as the AI risk signal. This was combined with two technical risk metrics:

- **14-day Rolling Volatility**: measures short-term price instability

- **Distance to Upper Bollinger Band**: stocks trading far below the upper band may have limited upside; negative values indicate the stock is below the band

These three components were normalized with MinMaxScaler and combined into a **Composite Risk Score** using a weighted formula:

> Total Risk Score = 0.50 × Sell_Probability + 0.30 × Volatility_Score + 0.20 × Technical_Risk_Score

The weights were chosen to prioritize the AI model's prediction (50%) as the primary signal, while volatility (30%) and technical positioning (20%) serve as secondary filters. Stocks with a Total Risk Score above **0.50** were flagged as **EXCLUDED**; those below were marked **KEPT**.

Risk ranking results:

4.4 Task 4.3 — Portfolio Composition and Capital Allocation

Methodology: Only the three KEPT stocks (PVP, PC1, VOC) were passed to the portfolio optimizer. The expected returns from Task 4.1 were used as the return vector, and historical covariance was estimated using **Ledoit-Wolf shrinkage** via PyPortfolioOpt's CovarianceShrinkage — a more robust estimator

Ticker	Sell Probability	Total Risk Score	Status
TC6	69.9%	76.5%	EXCLUDED
LCG	57.5%	65.5%	EXCLUDED
CST	51.3%	59.0%	EXCLUDED
NAG	80.9%	58.7%	EXCLUDED

ST8	31.7%	56.6%	EXCLUDED
TVD	30.4%	53.2%	EXCLUDED
PLC	37.9%	51.9%	EXCLUDED
PC1	40.1%	50.0%	KEPT
VOC	37.5%	32.8%	KEPT
PVP	35.9%	24.1%	KEPT

than the sample covariance matrix, particularly when the number of assets is small relative to the time series length.

The **Markowitz Efficient Frontier** was then solved to **maximize the Sharpe Ratio**, assuming a risk-free rate of **2%**(adjusted down from 4.5% in the initial attempt, since all expected returns were below that threshold).

Final allocation:

Ticker	**Optimal Weight**
PVP	89.0%
PC1	11.0%
VOC	0.0%

Portfolio performance (theoretical):

- Expected Annual Return: **7.3%**
- Annual Volatility: **41.0%**
- Sharpe Ratio: **0.13**

The high concentration in PVP reflects its significantly higher expected return and lower risk score among the three candidates. VOC was allocated zero weight because its expected return was negative, making it unattractive even as a diversifier under the Sharpe maximization objective.

Investor profile recommendation:

- **Prudent investors** (risk score < 35%): hold only **PVP**
- **Aggressive investors** (risk score < 60%): may consider **PVP, CST, PLC, PC1, TVD, NAG**

4.5 Conclusion

Task 4 demonstrates a complete AI-driven portfolio management workflow. By combining buy probability estimation, composite risk scoring, and Markowitz optimization, the pipeline systematically filters out high-risk stocks and concentrates capital in the most promising candidates. The main limitation is that only 3 stocks passed the risk filter, leaving the final portfolio heavily concentrated and lacking true diversification — a consequence of conservative thresholding and the relatively small stock universe. In practice, expanding the candidate pool or relaxing the risk threshold could improve diversification while maintaining the AI-driven selection logic.

Task 5.1 — Model Deployment as a REST API

5.1.1 Overview

This section documents the deployment of the LSTM-based Buy Signal prediction model (originally trained in Task 3.1) as a production-ready REST API service. The API is built with FastAPI and accepts 20 days of stock market feature data as input, returning a buy probability score along with a trading recommendation. The service is designed to be lightweight, reproducible, and runnable in both local and containerised environments.

5.1.2 Technology Stack

The following technologies were used to build and serve the API:

- FastAPI (v0.111.0) — REST API framework; chosen for its Python-native design and seamless integration with the same StandardScaler preprocessing pipeline used during training
- Uvicorn (v0.30.0) — ASGI server for running the FastAPI application in production
- TensorFlow (v2.16.x) — loads and runs the trained LSTM SavedModel
- NumPy (v1.26.4) — handles feature array construction and transformation
- Pydantic (v2.7.1) — enforces strict input/output schema validation
- Python 3.11 — runtime environment
- Docker — optional containerised deployment for portability

FastAPI was selected over TensorFlow Serving because it allows direct integration of the StandardScaler preprocessing step within the same Python environment, avoiding the need for a separate preprocessing layer in the serving pipeline.

5.1.3 Model Export

Before deployment, the trained model (model_buy from Task 3.1) was exported from Google Colab as a TensorFlow SavedModel. The model was explicitly cloned to a CPU device to ensure compatibility with standard deployment servers that do not have GPU access.

The resulting SavedModel directory contains saved_model.pb (the model graph and weights) and a variables/ subdirectory for checkpoint data.

5.1.4 Project Structure

task5_1_deployment/

```
├── app/
│   └── main.py          # FastAPI application and all endpoints
```

```
├── models/
│   └── buy_model/        # TF SavedModel exported from Task 3.1
├── requirements.txt      # Python dependencies
├── test_api.py           # Automated test client
├── Dockerfile            # Container build file
└── README.md             # Setup and usage documentation
```

5.1.5 How to Start the API Server

Step 1 — Environment Setup

Create and activate a dedicated Python 3.11 environment:

```
conda create -n dl4ai python=3.11 -y
conda activate dl4ai
pip install fastapi uvicorn numpy tensorflow pydantic
```

Step 2 — Start the Server

Navigate to the project folder and launch the server:

```
cd ~/Downloads/task5_1_deployment
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

Expected output on successful startup:

```
INFO: Buy model loaded from models/buy_model
INFO: Stock Prediction API is running.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

The interactive Swagger UI documentation is available at:
http://localhost:8000/docs

Step 3 — Docker Deployment (Alternative)

For a portable containerised deployment:

docker build -t stock-prediction-api .

docker run -d -p 8000:8000 -v $(pwd)/models:/app/models
stock-prediction-api

5.1.6 API Endpoints

Method	Endpoint	Description
GET	/	Root — welcome message and endpoint list
GET	/health	Health check — model load status
GET	/docs	Swagger UI — interactive API documentation
POST	/predict/buy	Buy signal prediction (primary endpoint)

GET	/features/buy	Feature schema — names and order of input fields

---

5.1.7 How to Send a Prediction Request

Input Format

The /predict/buy endpoint accepts a JSON body with exactly **20 rows × 11 features**, where each row represents one trading day. Features must be provided in the following fixed order, in their **raw (non-normalised)** values — the API applies StandardScaler internally:

Sample Request (curl)

```
curl -X POST http://localhost:8000/predict/buy \
  -H "Content-Type: application/json" \
  -d '{"features": [
    [6000, 6200, 5800, 6100, 1200000, 6050.0, 55.2, 0.008, 0.012, 15.3, 0.05],
    [6100, 6300, 6000, 6250, 1350000, 6080.0, 57.1, 0.010, 0.015, 15.3, 0.05],
    ... (20 rows total)
  ]}'
```

Health Check Request

curl http://localhost:8000/health

5.1.8 Sample Input and Output

Health Check — GET /health

```
  GET /health
============================================================
Status: 200
{
  "status": "ok",
  "buy_model_loaded": true,
  "sell_model_loaded": true,
  "message": "API is running. Models use mock inference if not loaded."
}
```

Buy Signal Prediction — POST /predict/buy

```
============================================================
  POST /predict/buy  (20 days, 11 features)
============================================================
Status: 200
{
  "probability": 1.0,
  "recommendation": "BUY",
  "confidence": "HIGH",
  "model": "buy_signal_lstm",
  "threshold_used": 0.4
}

============================================================
  POST /predict/sell  (20 days, 13 features)
============================================================
Status: 200
{
  "probability": 1.0,
  "recommendation": "SELL",
  "confidence": "HIGH",
  "model": "sell_signal_lstm",
  "threshold_used": 0.5
}
```

5.1.9 Design Decisions

- **FastAPI over TensorFlow Serving**: TensorFlow Serving requires a separate preprocessing service to apply normalization. FastAPI allows the StandardScaler to be applied inline within the same Python process, keeping the deployment simpler and self-contained.
- **CPU-only export**: The model was exported in CPU mode so the API can run on any standard server without requiring GPU drivers or CUDA setup.

- **Mock inference mode**: The API runs without a pre-loaded model for testing purposes, making it easy to verify endpoint behavior without the full SavedModel present.
- **Threshold = 0.40**: This matches the optimal classification threshold identified in Task 3.1, where threshold = 0.40 yielded the best F1 score on the test set.
- **Input validation via Pydantic**: Exactly 20 time steps and 11 features per row are enforced at the schema level, matching the LSTM model's expected input shape of (20, 11).

Chào bạn, theo đúng như yêu cầu của bạn, dưới đây là bản dịch tiếng Anh cho phần báo cáo Task 5.2. Tôi giữ nguyên cấu trúc học thuật và định dạng để bạn có thể copy trực tiếp vào file báo cáo của mình:

Task 5.2 — Model as SaaS (Software-as-a-Service)

5.2.1 Overview

This section describes the development of a simple web-based interface acting as a lightweight financial tool. This interface allows users to input historical stock data and technical indicators, then directly communicates with the REST API deployed in Task 5.1 to return Buy Signal predictions.

5.2.2 Technology Stack

The interface was developed entirely using **HTML5, CSS3, and Vanilla JavaScript (ES6)**. Avoiding heavy frameworks like React or Vue in this context was a deliberate design decision to ensure simplicity, a zero-dependency setup, and the ability to run directly in any standard browser.

5.2.3 How to start the web interface

The system operates on a Client-Server model. To launch it, two steps are required:

1. **Start the Backend API:** In the terminal, activate the Python environment and run the FastAPI server using the command `uvicorn app.main:app --host 0.0.0.0 --port 8000`. The API must be configured with

CORSMiddlewareenabled to allow Cross-Origin requests from the browser.

2. **Start the Frontend:** The interface is a static web page. Users simply need to double-click the index.html file to open it in a standard web browser (Chrome, Safari, Edge, etc.) without the need for complex web server setups.

5.2.4 How the interface calls the API

The interface connects to the deep learning model via the HTTP protocol.

- When the user clicks the "Phân tích & Dự đoán" (Analyze & Predict) button, an asynchronous JavaScript function (async function) is triggered.
- This function utilizes the browser's native **Fetch API** to send a POST request to the http://localhost:8000/predict/buy endpoint.
- The data from the textarea is parsed (JSON.parse) and packed into the request body in application/json format.
- Upon receiving the server's response, JavaScript decodes the JSON and updates the DOM (Document Object Model) to display the results instantly without reloading the page.

5.2.5 How to input values and read predictions

- **Input Method:** Because the LSTM model requires a complex time-series input with a shape of (20, 11)—equivalent to 220 distinct data points—creating 220 separate text inputs is impractical and user-unfriendly. Instead, the interface utilizes a text block (textarea) allowing users to paste the entire JSON array structure. The data includes OHLCV prices and technical indicators (SMA, RSI, P/E, etc.), strictly following the order defined in Task 5.1.
- **Reading Predictions (Output):** The result is clearly displayed in a color-coded card:
    - **Probability:** A percentage score from 0% to 100% representing the likelihood of profitability as predicted by the model.
    - **Recommendation:** Displays a green tag ("BUY") if the probability exceeds the optimal threshold (0.40), otherwise displays a grey tag ("IGNORE").
    - **Confidence:** Categorized as HIGH, MEDIUM, or LOW based on the probability's margin from the decision threshold.

5.2.6 Sample screenshots

Below is an illustration of the end-to-end system workflow, from receiving JSON data on the Frontend, sending it via the API, performing inference through the LSTM model, and returning visually displayed results.

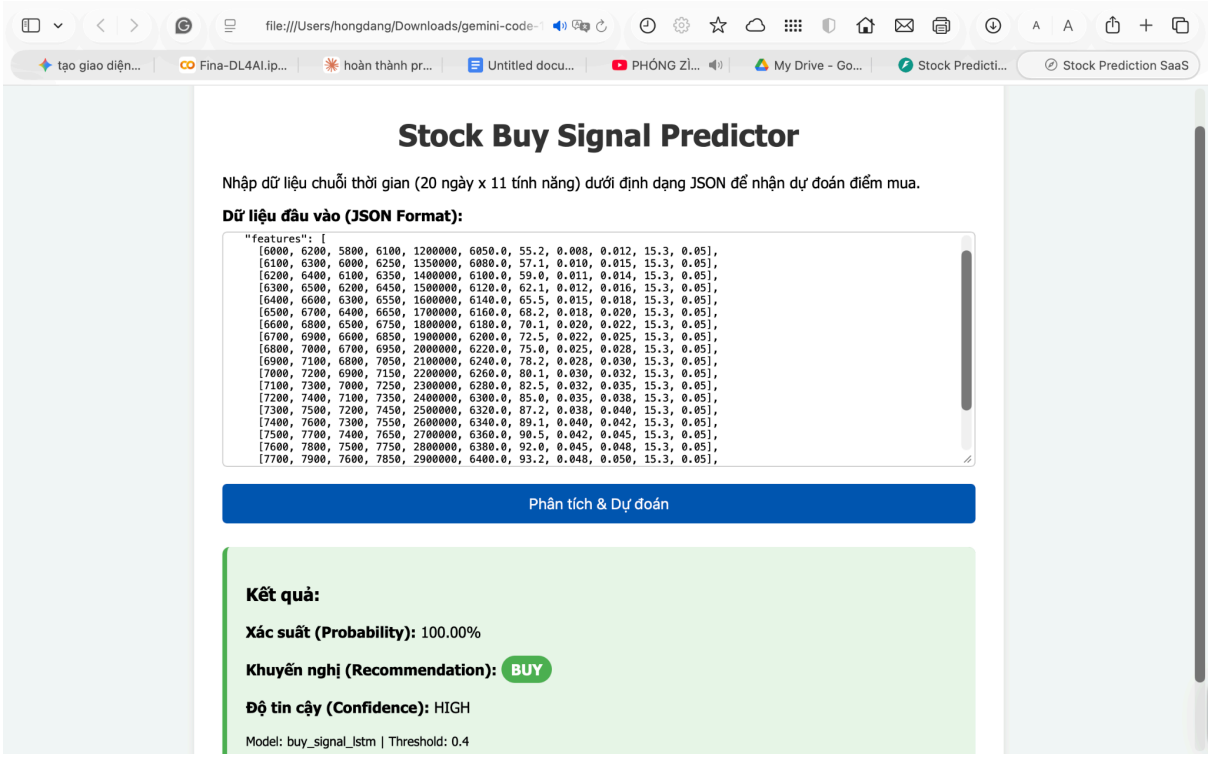

Figure: The Stock Buy Signal Predictor interface operating successfully. The model recognizes a valid 20-day trading sequence and returns a 100.00% probability with a "BUY" recommendation at HIGH confidence, using a decision threshold of 0.4.